OptiTAC

Compiler Optimization Analysis, Explanation and Validation

PROJECT REPORT

Team member	Register number
Archit Nigam	24BCI0230
Nettem Ronith Kumar	24BCT0285
Pranshu Gupta	24BCE2099

October 2026

Abstract

OptiTAC is a local compiler laboratory for studying how source programs become intermediate code and how optimization changes that code. The project began with a MiniC compiler and now provides a documented scalar C/C++ subset through six stages: Tokens, Syntax, Types, Flow & IR, Optimize and Results. Each stage retains artifacts that students can inspect alongside the source program.

The compiler produces typed three-address intermediate representation (IR), constructs control-flow information and applies four optimization passes. A bounded beam search explores pass orders. A bundled random forest ranks the passes using predicted reductions in a static IR cost proxy. The model guides search expansion, while concrete execution comparisons determine whether a candidate is eligible for selection.

Validation compares the AST reference, original IR and selected IR using complete stdout and typed return values. The system adds generated integer probes around supported affine comparison boundaries. When a rejected transformation exposes a disagreement, a bounded reducer attempts to simplify the input and shorten its ordered pass sequence. A further audit builds a witness matrix and selects a compact suite while retaining structural anchors.

Repository experiments report detection of 110 of 120 deliberately seeded faults using targeted probes. An anchored suite retained the same observed detection with 242 replay inputs instead of 960. These results describe a designed synthetic corpus, rather than arbitrary C/C++ programs. Finite observations do not establish semantic equivalence, and the static-cost measure does not establish runtime speedup.

Keywords

Compiler optimization, three-address code, dataflow analysis, phase ordering, random forest, boundary testing, mutation testing, failure reduction.

Report basis

This report describes the active Compiler Lab implementation and the recorded evaluation in the project repository as of 28 October 2026. Team details follow the supplied Review 1 presentation. Evaluation figures are attributed to saved project records; they are not presented as a fresh experimental run.

1. Introduction

1.1 Background

Compiler teaching introduces lexical analysis, parsing, semantic checks and optimization as distinct topics. A student can understand each definition yet still struggle to connect a source expression with its lowered instructions or to explain why a later pass becomes useful. A final optimized listing alone gives little evidence about the intermediate decisions.

OptiTAC keeps these stages in one workflow. Users write a supported C or C++ program, supply independent stdin cases and inspect the artifacts from a single analysis. Selecting tokens links the lexical output to source locations. The syntax tree exposes precedence and nesting. Flow and IR views connect the source to control-flow blocks and live-variable information.

1.2 Problem statement

The project aims to implement a compiler laboratory that makes intermediate transformations inspectable, explains interactions between optimization passes and checks the resulting behavior on defined inputs. It also addresses the weakness of validating an optimization only on a few convenient examples, which may miss a wrong rewrite on an untested branch.

1.3 Objectives

The implementation objectives are to build a typed compiler pipeline, explore bounded optimization orders, retain before-and-after transformation evidence and compare source and IR observations. The testing extension aims to propose relevant boundary inputs and preserve replayable counterexamples. The witness audit examines whether a smaller collection of tests can retain useful evidence beyond coverage of the known local challenges.

1.4 Scope

The active tool is an educational interpreter and optimization laboratory for a documented scalar subset. An optional local GCC/G++ service provides native diagnostics and compiler artifacts for broader syntax. It does not run submitted native programs. The project does not claim a complete C/C++ implementation, a formal equivalence checker or an established improvement in student learning outcomes.

2. Technical Context and Project Contribution

2.1 Optimization and explanation

Constant propagation, constant folding, reachability analysis and liveness are established compiler techniques. LLVM documents analysis and transformation passes as part of its compiler infrastructure [6]. OptiTAC uses established methods in a smaller setting where users can inspect the intermediate code and the reason recorded for each transformation.

The order of passes matters because one pass can expose an opportunity for another. Propagating a known value can produce a constant expression. Folding that expression produces another constant that can travel further through the program. If a branch condition becomes constant, reachability cleanup may remove a block. The project presents these relationships through logs and replays that omit one pass from a selected route.

2.2 Native compilation and validation

GCC distinguishes preprocessing, compilation, assembly and linking [7]. OptiTAC keeps native checks separate from visual execution. A native compiler may accept a class or a standard container even when the educational interpreter cannot execute it. The application reports these two outcomes separately rather than inventing a visual execution result.

Alive2 is a related system for verification of LLVM transformations [8]. OptiTAC provides a narrower assurance level: it compares concrete, bounded observations on a finite input set. A passing comparison therefore supports the checked cases, rather than proving equivalence for every possible input.

2.3 Contribution

The project contribution is the integration of inspectable compiler artifacts, ML-ranked pass exploration, conservative fallback, targeted test generation and reduced failure evidence. The witness extension adds an explicit account of which controlled faults each input distinguishes, and preserves selected source-structure obligations during suite compression.

These individual techniques are established. The repository treats the combined mechanism as a research candidate and does not claim a first implementation or publication acceptance. Independent benchmarks and stronger comparisons remain necessary.

3. Requirements and System Scope

Area	Implemented requirement
Source analysis	Positioned tokens, syntax tree, lexical scopes and type checks
Intermediate representation	Typed TAC, control-flow blocks and live-variable sets
Optimization	Four passes, bounded route search and detailed transformation logs
Validation	AST/original/selected IR comparisons on supplied and generated inputs
Diagnostics	Rejected-candidate replay and bounded input/pass reduction
Evidence	Witness matrix, selected suite, audit status and JSON export

3.1 Language contract

Supported constructs include scalar int, float, double, bool and char values, declarations, lexical scopes, assignments, arithmetic and comparison expressions, logical and bitwise operators, casts, branches, loops, break/continue, scalar functions and bounded recursion. Selected C input/output functions and successful C++ stream operations have explicit adapters [2].

The visual interpreter excludes pointers, arrays, classes, templates, general preprocessing and arbitrary libraries. Integers follow checked signed 32-bit semantics. Integer division truncates toward zero. Float values round through binary32 and double values use binary64. Invalid shifts, division by zero, uninitialized reads and unsupported constructs produce diagnostics.

3.2 Operational requirements

The browser interface must remain responsive during analysis. A Web Worker performs the visual analysis and supports cancellation. Evidence export must preserve outcomes and audit details. No API key, remote model service or model download is required. Optional GCC/G++ availability affects native checks and dumps, rather than the availability of the visual analysis pipeline.

4. Architecture and Data Flow

Component	Responsibility
React interface	Source editing, six stage views, replay and evidence inspection
Analysis worker	Orchestrates compiler stages without blocking the interface
Compiler core	Lexing, parsing, semantics, typed lowering and interpretation
Optimization search	Pass expansion, observation guards and measured-cost selection
ML advisor	Offline-trained pass-ranking predictions and fallback
Research audit	Probe generation, failure reduction and witness-suite selection
Local native service	GCC/G++ diagnostics and dumps in compile-only mode

4.1 Analysis sequence

The user chooses a dialect, provides a program and enters independent stdin cases. Lexical analysis produces tokens with positions. Parsing builds the AST. Semantic analysis resolves lexical bindings and checks types. Lowering then produces typed IR and control-flow artifacts. Failed stages retain earlier artifacts so that users can inspect where analysis stopped [1, 2].

Optimization begins only within the supported visual contract. Candidate transformations carry logs, measured static cost and validation outcomes. The selected route remains available for intermediate-pass inspection. Results compare the AST reference with original and selected IR, while JSON export retains the evidence needed for replay.

4.2 Separation of native artifacts

The optional localhost service invokes GCC/G++ for C11/C++14 checks and selected tree, GIMPLE, CFG and optimized-GIMPLE dumps. It applies documented source, time and output limits. Native availability and native syntax rejection are explicit states. Compilation limits do not constitute an operating-system security sandbox [2].

5. Compiler Pipeline and Worked Example

The following scalar program illustrates the relationship between source, syntax and TAC. The listings are teaching examples, not verbatim exports from the current typed IR serializer.

```
int main() {
    int a = 4;
    int b = a * 2 + 1;
    int c = b + 10;
    return c;
}
```

5.1 Tokens, structure and types

For the declaration of b, the lexer identifies the type keyword, identifier, assignment, operands and operators. The parser represents multiplication beneath addition, preserving precedence. Semantic analysis checks that a is declared, resolves its binding and verifies that the expression can initialize b.

5.2 Three-address form

```
a = 4
t1 = a * 2
t2 = t1 + 1
b = t2
t3 = b + 10
c = t3
return c
```

Each arithmetic instruction has a simple operation and explicit operands. Propagation replaces a with 4. Folding produces t1 = 8, after which the next expression becomes 8 + 1. Continuing the same reasoning yields b = 9, c = 19 and return 19. The active compiler retains type and control-flow information beyond this simplified listing.

6. Optimization and Route Explanation

Pass	Behavior and restriction
Conditional constants	Worklist propagation joins facts at incoming edges and folds typed expressions.
Reachability	Removes unreachable instructions after control-flow discovery.
Dead temporaries	Uses backward liveness and removes safe unused literals/copies while retaining effects and potentially failing operations.
Jump cleanup	Simplifies redundant control-flow instructions.

6.1 Bounded search

The search uses beam width 3, depth 4 and at most 32 distinct transformed candidates. A documented static-cost proxy assigns cost 3 to compute instructions, 2 to branches, 1 to other executable instructions and 0 to structural instructions. Candidates compete by measured cost after eligibility checks [2].

This cost is useful for comparing IR within the laboratory. It does not measure execution time, energy use or machine-code performance. A smaller static listing can still behave differently at runtime, and the project does not infer runtime speedup from this proxy.

6.2 Logs and dependency analysis

Transformation logs record the changed instruction, its previous and revised form, the pass and an explanation. Intermediate-pass navigation allows users to inspect how the chosen route evolves. A counterfactual replay omits one pass from that fixed route and applies the remainder. It helps explain interactions, but does not perform a new search for the globally best route.

In the worked example, propagation enables folding and the folded result enables further propagation. These are concrete local relationships. They do not imply that the same route is best for every program or that omitting a pass proves a global causal claim.

7. ML Guidance

7.1 Model and training target

A bundled multi-output random forest regressor predicts the static-cost reduction of each of the four passes. The active dataset contains 1,376 unique generated C/C++ programs and 4,784 measured IR states. Twenty-seven static IR features describe control flow, expressions, temporaries, effects, calls and unreachable instructions [3].

The target for a pass is (cost before - cost after) / max(1, cost before). Training runs the actual pass to obtain its label. Runtime feature extraction does not execute candidate passes to obtain their measured savings. Training is an offline development command, rather than an action performed on user submissions.

7.2 Split and fallback

The dataset uses 80 structural template groups split into 50 training, 10 validation, 10 calibration and 10 test groups. Related numeric and language variants remain together. The state counts are 2,972, 612, 652 and 548. Broad program families occur across partitions, so the holdout measures related synthetic structures rather than arbitrary-program generalization.

If a feature falls outside the training minimum/maximum envelope, the system uses default ordering. Missing or incompatible artifacts also trigger fallback. The model may still display an unused prediction, with its status. An empirical calibration band represents observed prediction error, not the probability that a transformation is safe.

7.3 Decision boundary

The model ranks search expansion. Measured static cost governs retained candidates, and defined execution comparisons determine eligibility. Disabling ML preserves the same guarded search with default ordering. A prediction alone never authorizes a transformation.

8. Validation and Failure Diagnosis

8.1 Observation checks

Validation compares complete stdout and typed return values for the AST reference, original IR and candidate IR. Up to 12 supplied input cases and eight eligible generated probes contribute to the checks. A source/original-IR disagreement prevents optimized acceptance. Undefined execution, unsupported behavior and exhausted execution limits do not count as successful validation [2, 4].

8.2 Comparison-targeted probes

The generator maps supported integer reads in main to input columns. A bounded forward analysis propagates affine relationships through copies, addition/subtraction and constant multiplication. For a supported comparison it proposes values around the equality boundary, varying one column while retaining the other seed values. Concrete reference execution determines whether a proposal is eligible.

The method does not solve complete path constraints. Nonlinear expressions and unsupported input formats limit it, and the remaining slots use fixed-value fallback. Probe executions use a 2,000-step bound. A candidate timeout is inconclusive and cannot validate a transformation.

8.3 Reproducible demonstration

The hidden-fault example returns 7 when n equals 137 and otherwise returns 2. A deliberately ineligible challenge changes 7 to 8. Supplied stdin 300 999 produces 2 in both versions and misses the fault. A targeted probe at 137 999 exposes the disagreement. Reduction retains 137 0 and removes unnecessary pass actions while preserving a defined mismatch [4].

The reducer considers at most three rejected candidates and 64 oracle calls per candidate. It simplifies integer values and deletes actions from the ordered sequence. Local irreducibility applies only to those operations when the budget permits completion. It does not mean globally smallest input or source-program reduction.

9. Witness Matrix and Compact Suites

9.1 Audit construction

The evidence audit builds a pool of up to 24 inputs from supplied cases and interleaved targeted, fixed and seeded-random proposals. It records bounded AST, original-IR and selected-IR observations. Up to ten controlled local mutations form challenges across literals, comparisons, arithmetic and output effects. Mutation timeouts remain inconclusive [5].

Each matrix row shows the challenges that one input distinguishes. A surviving challenge remains unresolved, since the system does not automatically establish equivalence or unreachability. The audit separately checks the actual selected IR. A defined disagreement on a wider input rejects that candidate, restores original IR and attempts failure reduction.

9.2 Structural anchors and selection

Selection retains source branch outcomes from two branch sites and center proposals from two affine comparison groups. These anchors protect some source-structure evidence while compressing the suite. Unobserved outcomes remain unknown, and a generated center is a heuristic obligation rather than solver-proven reachability.

The exact bounded selector chooses at most eight inputs. Its priorities are to maximize covered anchors, then distinguished challenges, then minimize suite size and finally minimize the observed replay-step proxy. A dynamic program retains the minimum cost for each anchor mask, challenge mask and selected count. Exactness applies to this finite matrix and objective.

9.3 Evidence accounting

The compact suite is a replay artifact. It does not replace the existing supplied-input guards. The report keeps planning cost, replay cost and unresolved challenges visible. This distinction matters because a small final suite can still require substantial work to construct, and complete coverage of known mutations can miss unseen fault operators.

10. Recorded Evaluation

The following figures come from the saved model card and research reports [3-5]. They describe separate experiments and should not be combined into a single accuracy score.

ML metric on 548 held-out states	Random forest	Mean baseline
Gain-ratio mean absolute error	0.00450	0.08993
Best-pass agreement, ties accepted	100.0%	59.9%

The 100% agreement applies only to this synthetic partition. A separate development set of 17 programs at three candidate budgets found lower static cost with ML in 3 of 51 paired comparisons, ties in 48 and no worse result. This is an integration check rather than an independent runtime benchmark.

Seeded-fault strategy	Detected / 120
Comparison-targeted + fallback	110
Fixed boundary values	2
Uniform random, default seed	5
Uniform random, mean over 32 seeds	8.41

Each strategy used eight probes per fixture. The corpus deliberately emphasizes supported affine boundaries. Its faults were injected into IR and are not discovered production compiler bugs. The four actual passes had zero disagreements in 3,840 recorded negative-control comparisons [4].

Operator-held-out suite	Detected / 120	Inputs
Targeted, eight per fixture	110	960
Mutation-only exact	49	144
Branch-only exact	98	230
Branch + boundary anchors, exact	110	242

Anchored exact and greedy selection tied on this pilot. The anchored suites used 74.8% fewer replay inputs than the targeted-eight baseline, with planning overhead of 11,556 interpreter executions and 269,760 steps. The recorded regression run on 28 October 2026 reports 290 passing tests, zero failures and zero skips, plus successful build, lint and browser checks [5].

11. Limitations, Future Work and Conclusion

11.1 Limitations

The interpreter implements a scalar subset and explicit numeric contract. Shared numeric primitives between AST and IR introduce a common-mode oracle risk: both can agree while sharing a defect. Native differential development tests add a separate check, but application-level native compilation remains compile-only.

The ML and fault-injection corpora are generated and structurally related. The targeted-testing corpus favors affine comparisons. Current results therefore do not establish broad production-compiler generalization. Finite probes cannot cover all paths, classify every surviving mutation or prove equivalence. The exact suite selector is exact only for its bounded recorded matrix.

11.2 Future work

The next evaluation should use independent external programs and stronger input-generation baselines under matched budgets. Runtime measurements would be needed before making performance claims. A more independent semantic oracle could reduce shared implementation risk. Language expansion should preserve explicit diagnostics for unsupported constructs and undefined behavior.

A controlled learning study could measure whether the stage views and dependency explanations improve student understanding. Tasks could ask participants to trace lowering, predict which pass becomes useful next and interpret a counterexample. The current implementation has not measured these learning outcomes.

11.3 Conclusion

OptiTAC provides a working connection between source analysis, typed intermediate code, optimization routes and validation evidence. ML guidance helps order exploration, while execution observations guard candidate selection. Targeted probes and reduced failures make missed behavior easier to inspect. The witness audit adds a record of validation quality and the cost of retaining it. The recorded experiments support these mechanisms within their stated scope, and define the remaining work needed for broader claims.

References

[1] OptiTAC project repository. README.md. Active Compiler Lab overview, execution instructions and implementation scope. Local project record, October 2026.

[2] OptiTAC project repository. docs/compiler-lab.md. Compiler Lab: implementation and novelty. Language contract, optimization search, validation and native compilation.

[3] OptiTAC project repository. docs/ml/model-card.md. ML-guided optimization with a boundary-input audit. Training corpus, grouped split, measured model results and limitations. Recorded 28 October 2026.

[4] OptiTAC project repository. docs/research/report.md and docs/research/evaluation.json. Targeted probes and failure reduction. Seeded-fault corpus, random replications, reducer results and limitations. Recorded 28 October 2026.

[5] OptiTAC project repository. docs/research/evidence-portfolios.md and docs/research/evidence-evaluation.json. Anchor-preserving witness portfolios. Operator-family holdout, replay counts, planning overhead and recorded regression verification. Recorded 28 October 2026.

[6] LLVM Project. LLVM Analysis and Transform Passes. https://llvm.org/docs/Passes.html. Accessed 29 October 2026.

[7] GNU Compiler Collection. Options Controlling the Kind of Output. https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html. Accessed 29 October 2026.

[8] AliveToolkit. Alive2: Automatic Verification of LLVM Optimizations. https://github.com/AliveToolkit/alive2. Accessed 29 October 2026.

Source presentation

OptiTAC_Review1.pptx, supplied by the project team. Used for the team names, register numbers, presentation dimensions and visual format. Historical implementation status in that presentation has been updated using the active repository records.